EMPIRICAL ANALYSIS OF ALGORITHMS

DAA Practical / Lab Activity

Objective: Implement the given algorithms, execute them for different values of N, record the number of times the basic operation executes and the actual execution time, and study the observed growth experimentally.

1. Algorithms to be Implemented

A. Linear Search

Write a program to search for a specified element in an array using Linear Search. Count the number of comparisons performed by the search.

B. Loop Analysis – Multiplicative Loop (starting from 0)

Write a program containing the loop: for(i = 0; i < n; i = i * 2). Observe and report its execution behaviour. Carefully handle initialization/update so that the program terminates correctly; explain your observation.

C. Loop Analysis – Multiplicative Loop (starting from 1)

Write a program containing the loop: for(i = 1; i < n; i = i * 2). Count how many times the loop body executes.

2. Important Experimental Requirement

Students must write the programs on their own. Do not use a ready-made implementation of the algorithm. The program must contain a counter for the selected basic operation so that the number of executions/comparisons can be recorded.

For every algorithm, use the following N values by changing N statically in the source code:

N = 10, 20, 100, 1000, 2000, 5000

3. Input Generation

For each value of N, create/generate an input of size N. Do not manually enter thousands of values. You may initialize the array programmatically. Keep the input-generation method consistent while comparing results.

4. What You Must Record

For each N, record the following two experimental measures:

- Number of times the selected basic operation executes (for example, comparisons).
- Actual execution time measured by the program in an appropriate unit such as microseconds or nanoseconds.

Use the same computer/environment for all experiments. Run the program separately for each N. For very small inputs, the measured time may be extremely small; report the value produced by your timing method rather than replacing it with a theoretical value.

5. Results Table

Prepare a separate table for each algorithm in the following format:

N	Number of Executions(Max)	Execution Time
10		
20		
100		
1000		
2000		
5000		

Important: The number of operations should come from your program's counter. Do not fill this column using the theoretical complexity formula.

6. Graphs to Prepare in MS Word/Excel

- Graph 1: N vs. Number of Basic-Operation Executions
- Graph 2: N vs. Actual Execution Time

Use a Line Chart in MS Word. Put N on the X-axis and the corresponding measured value on the Y-axis. Give every graph a suitable title and label both axes with units where applicable.

7. Analysis / Observation

After the two graphs for each algorithm, write 3–5 sentences explaining the observed trend. Compare the experimental growth with the theoretical time complexity taught in class. Mention any noticeable variation in execution time and possible reasons such as system load, compiler optimization, or very small measurement intervals.

8. Suggested Submission Structure

1. Algorithm / program code for each of the five programs.
2. Output screenshots showing the measured values.
3. Results table for each algorithm.
4. Line graph: N vs. Number of Basic-Operation Executions.
5. Line graph: N vs. Execution Time.
6. Short observation/conclusion for each algorithm.

9. Submission

Compile all programs, output tables, graphs, and observations into a single MS Word file.

Submit the completed file using the attached submission link.

File naming format: Name_RollNo_EA.pdf

Google Form Link :-

https://docs.google.com/forms/d/e/1FAIpQLSdDIA-bC1FxmESrokfYTF8mhOuJy06R77bmN537fTqX_R9AYA/viewform?usp=dialog

***** *Happy Analysis* *****